

CS 650 – Full Stack Application Develop

**4-2 Activity: Secure Handling of Secrets and User Data**

Malgorzata Debska

Southern New Hampshire University

Brian Enochson

July 28, 2026

## **Module Four Activity: Secure Handling of Secrets and User Data**

Security is one of the most important aspects of full stack application development because users trust applications to protect their personal information. Even small programming mistakes can expose sensitive data and create opportunities for attackers to gain unauthorized access. During this activity, I explored an intentionally vulnerable web application to identify common security weaknesses and understand how they could affect users and businesses. The application contained examples of Insecure Direct Object References (IDOR), SQL Injection, and Cross-Site Scripting (XSS). By examining these vulnerabilities, I gained a better understanding of how insecure coding practices can expose sensitive information and how secure development techniques can prevent these risks.

### **Identifying Vulnerabilities That Expose Sensitive User Information**

#### **Insecure Direct Object Reference (IDOR)**

The first vulnerability I identified was an Insecure Direct Object Reference (IDOR). This vulnerability exposed another user's account information simply by modifying the identifier in the application's URL. Instead of verifying whether the authenticated user had permission to access the requested account, the application displayed information belonging to someone else. This issue exposed private user data that should only be available to the account owner. The vulnerability was observed after manually changing the account ID within the browser address bar and receiving another user's information without any authorization check.

## Evidence:

localhost

Activity: Secure Handlin... Codio - Full Stack Appli... Mail - Debska, Malgorz... Assignment - CS 650 .d... Module Four Activity Se...

### SecureBank Internal Portal

CTF Security Exercise — Find the 3 hidden flags

#### 1. Account Details

Logged in as **alice** (Account #1). View your account details below.

[View My Account](#)

#### 2. User Directory

Search for employees by username. Try: alice, bob, charlie, diana, eve

**Username**

[Search](#)

#### 3. Welcome Greeting

Enter your name to receive a personalized greeting.

**Your Name**

[Greet](#)

SecureBank Internal Portal

localhost:3000/api/account/1

localhost:3000

# SecureBank Internal Portal

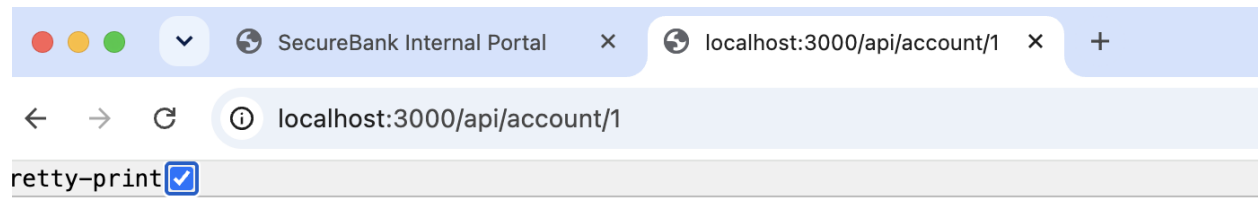
CTF Security Exercise — Find the 3 hidden flags

## 1. Account Details

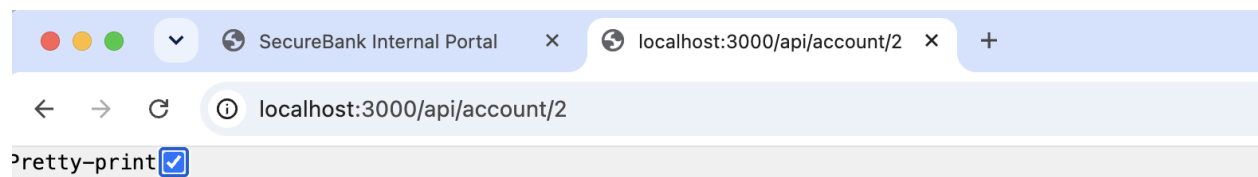
Logged in as **alice** (Account #1). View your account details below.

View My Account

|            |                          |
|------------|--------------------------|
| Account ID | 1                        |
| Username   | alice                    |
| Email      | alice@securebank.com     |
| Balance    | \$12450.00               |
| Notes      | Primary checking account |



```
"id": 1,  
"username": "alice",  
"email": "alice@securebank.com",  
"balance": 12450,  
"notes": "Primary checking account"
```



```
{  
  "id": 2,  
  "username": "bob",  
  "email": "bob@securebank.com",  
  "balance": 8200.5,  
  "notes": "Savings account"  
}
```

## SQL Injection

The second vulnerability involved SQL Injection within the application's search functionality. The application accepted user input and inserted it directly into an SQL query without properly validating or sanitizing the data. By entering specially crafted SQL commands into the search field, it was possible to retrieve information from database tables that should never be publicly accessible. This vulnerability exposed sensitive database information and demonstrated how attackers could potentially access confidential customer records, login credentials, or other protected information stored in the database.

### Evidence:

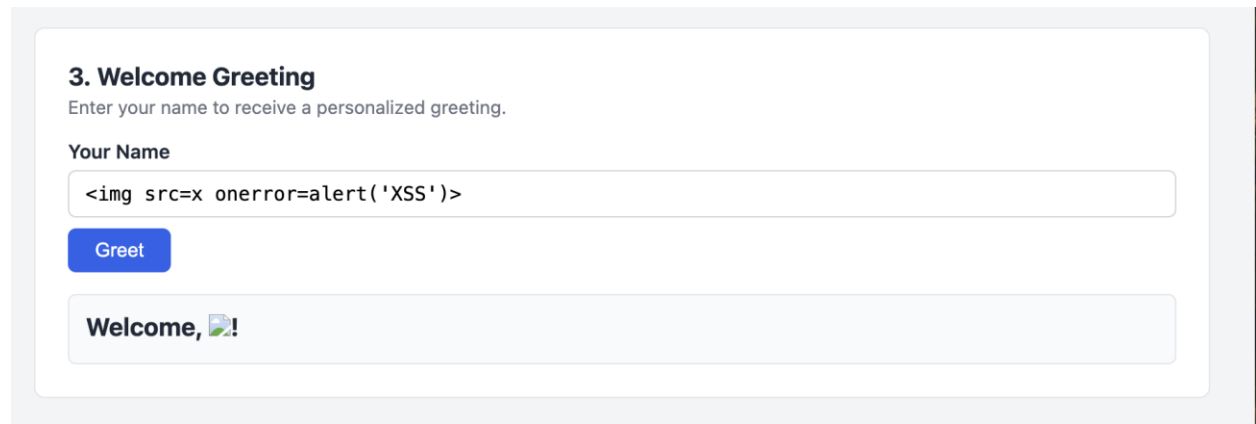
The screenshot shows a web browser with two tabs: 'SecureBank Internal Portal' and 'localhost:3000/api/account/1'. The address bar shows 'localhost:3000'. The page content is titled '2. User Directory' and includes a search prompt: 'Search for employees by username. Try: alice, bob, charlie, diana, eve'. Below this is a 'Username' input field containing the SQL injection payload: `' OR '1'='1`. A blue 'Search' button is positioned below the input field. The search results are displayed in a table with three columns: 'id', 'username', and 'email'.

| id | username | email                  |
|----|----------|------------------------|
| 1  | alice    | alice@securebank.com   |
| 2  | bob      | bob@securebank.com     |
| 3  | charlie  | charlie@securebank.com |
| 4  | diana    | diana@securebank.com   |
| 5  | eve      | eve@securebank.com     |

## Cross-Site Scripting (XSS)

The third vulnerability identified was Cross-Site Scripting (XSS). The application reflected user input to the browser without properly encoding or escaping special characters. Because of this, JavaScript code entered in an input field was executed directly within the webpage. Although the activity demonstrated a simple JavaScript alert, a real attacker could use this vulnerability to steal session cookies, capture authentication tokens, redirect users to malicious websites, or impersonate legitimate users.

### Evidence:



**3. Welcome Greeting**  
Enter your name to receive a personalized greeting.

Your Name

`<img src=x onerror=alert('XSS')>`

Greet

Welcome, !

## Common Patterns Across the Vulnerabilities

While each vulnerability affected a different part of the application, they all resulted from similar secure coding failures. The application trusted user input without sufficient validation, relied on client-side information instead of performing proper server-side authorization, and failed to sanitize or encode data before processing or displaying it. These common weaknesses highlight the importance of following secure-by-design principles throughout the software development

lifecycle. Small oversights in handling user input or validating requests can create multiple opportunities for attackers to expose sensitive information.

## **Relative Risk Assessment**

Among the vulnerabilities discovered, SQL Injection represents the greatest overall security risk. Because the vulnerability interacts directly with the application database, a successful attack could expose thousands of customer records, modify critical business information, or even completely compromise the database. In many real-world attacks, SQL Injection has resulted in large-scale data breaches affecting millions of users. The people most affected by SQL Injection include customers whose information is stored in the database, system administrators responsible for maintaining the application, and the organization itself. The consequences may include financial losses, legal penalties, regulatory violations, and significant damage to customer trust.

The IDOR vulnerability presents a moderate level of risk because it allows unauthorized users to view information belonging to other customers. While it generally affects individual accounts rather than the entire database, attackers can automate these requests to collect large amounts of sensitive information over time.

The XSS vulnerability also presents a moderate risk. Although it does not directly expose server-side data, it allows malicious JavaScript to execute within a user's browser. This can lead to stolen session cookies, account hijacking, phishing attacks, or malicious actions performed on behalf of legitimate users.

## **Secure Handling Practices for Protecting Sensitive User Information**

Each vulnerability can be significantly reduced by following well-established secure development practices. To prevent IDOR, the application should perform authorization checks on the server before returning any requested resource. Rather than trusting identifiers provided by the browser, the server should verify that the authenticated user has permission to access the requested account or record. This prevents unauthorized users from viewing or modifying information that does not belong to them. To prevent SQL Injection, developers should always use parameterized queries or prepared statements instead of building SQL commands by concatenating user input. Prepared statements ensure that user input is treated strictly as data rather than executable SQL code. Input validation should also be implemented to reject unexpected or malicious values before they reach the database. To prevent Cross-Site Scripting, all user-generated content should be properly encoded before being displayed in the browser. Modern web frameworks typically include automatic output encoding, and developers should ensure these protections remain enabled. Additional safeguards such as Content Security Policy (CSP), HttpOnly cookies, and input validation provide additional layers of defense against browser-based attacks.

## **Why These Vulnerabilities Should Be Addressed**

Leaving these vulnerabilities unresolved could have serious consequences for both users and the organization. Sensitive customer information could be stolen, user accounts could be compromised, and confidential business data could be exposed or manipulated. Beyond the

immediate technical risks, successful attacks often lead to financial losses, legal consequences, regulatory penalties, and long-term damage to an organization's reputation.

Among the vulnerabilities identified, SQL Injection should receive the highest priority because of its ability to expose or modify an application's entire database. Addressing this vulnerability first significantly reduces the overall risk to the organization. The IDOR vulnerability should be prioritized next because it directly exposes private user information and violates access control requirements. Finally, the XSS vulnerability should also be addressed promptly to protect users from account hijacking, phishing attacks, and malicious browser activity. Implementing secure coding practices early in the development process, organizations can greatly reduce the likelihood of security incidents while protecting customer information and maintaining user trust.